Lecture 5
Convolutional Neural Networks
AI for Geometry — MM845

Edward Hirst

Instituto de Matemática, Estatística e Computação Científica
Universidade Estadual de Campinas

ehirst@unicamp.br

Agosto 2026

Overview

1. Why Convolutions?
2. Anatomy of a CNN
3. What CNNs Learn
4. CNNs for Geometric Data
5. Summary

Recap (Lecture 4): NNs (MLPs) are universal approximators trained by backpropagation.
Guiding question: *How is a symmetry built into a hypothesis space (exactly)?*

Grid Data and the Failure Mode of MLPs

Data as functions on a grid

An image is a map

$$u : \underbrace{\Omega}_{\substack{\text{finite window of sites} \\ \Omega \subset \mathbb{Z}^2}} \longrightarrow \underbrace{\mathbb{R}^c}_{\substack{\text{the } c \text{ numbers} \\ \text{stored at one site}}}$$

$\to$ The lattice $\mathbb{Z}^2$ is both the set of sites and the group translating them.
$\to$ Ω is the finite part we sample, say $\{0, \dots, 255\}^2$.
$\to$ The **channel** count c is the dimension of the site's value: $c = 1$ a scalar field, $c = 3$ RGB.
$\to$ So the data space is $\mathbb{R}^{|\Omega| \times c}$: discretised sections of a trivial rank-c bundle over the grid — curvature fields on a chart, heightmaps of surfaces, matrices of combinatorial data all live here.

Feeding a 256×256 image to an MLP by flattening:

- First layer with 1000 hidden units: $\sim 6.5 \times 10^7$ parameters — for *one* layer.
- Destroys the grid structure: pixel (i, j) and $(i, j{+}1)$ are as unrelated as opposite corners.
- No translation structure: the network must re-learn the same feature at every location independently.

Two priors worth building in

Locality: meaningful features are local (edges, corners, curvature).
Translation symmetry: a feature is the same feature wherever it occurs.
...*convolution* is the unique linear structure implementing both.

Convolution and the Equivariance Theorem

Discrete convolution (cross-correlation, as implemented)

For signal $u : \mathbb{Z}^2 \to \mathbb{R}$ and kernel w supported on a small window K (e.g. 3×3):

$$(w \star u)(p) \;=\; \sum_{q \in K} w(q)\, u(p+q).$$

Theorem (translation equivariance characterises convolution)

Let $T_v u(p) = u(p - v)$ be translation. A linear map Φ on signals satisfies

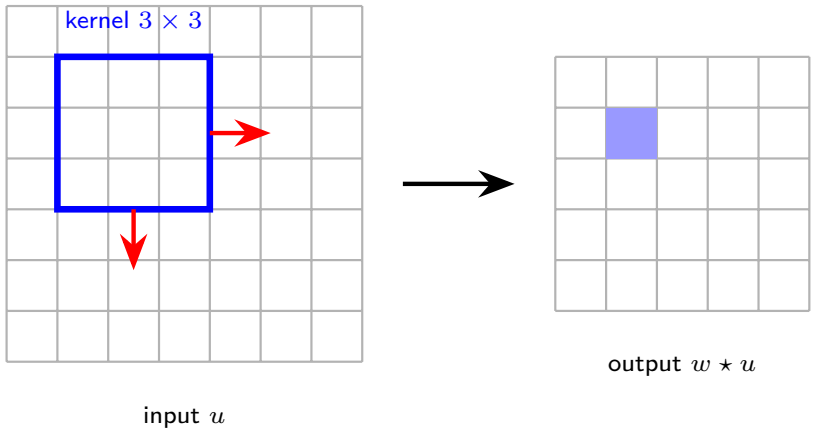
$$\Phi \circ T_v = T_v \circ \Phi \quad \forall v \in \mathbb{Z}^2$$

if and only if Φ is a convolution, therefore choosing convolution makes translation equivariant.
$\to$ Using a convolution as the architecture layer enforces this translation symmetry into $\mathcal{H}$.
(This idea returns for general groups in Lecture 10)

- Consequence: **weight sharing**. One small kernel is applied everywhere: parameters drop from 10^7 to ~ 10 per filter, and the translation prior is exact, not learned.
- Continuous analogue: translation-invariant operators are Fourier multipliers — convolution theorem; CNNs inherit a spectral theory (used in Lecture 13's neural operators).

The Convolutional Layer

- **Channels**: layer maps c_{in} input channels to c_{out} output channels (every output channel reads *all* input channels). In bundle terminology:
  $\Phi : \Gamma\big(\Omega, \Omega \times \mathbb{R}^{c_{\text{in}}}\big) \longrightarrow \Gamma\big(\Omega, \Omega \times \mathbb{R}^{c_{\text{out}}}\big)$,
  ...so the kernel is Hom-valued:
  $w : K \to \mathrm{Hom}(\mathbb{R}^{c_{\text{in}}}, \mathbb{R}^{c_{\text{out}}})$.
- **Parameter count**: $c_{\text{in}} \times c_{\text{out}} \times k^2 + c_{\text{out}}$
  $(\Phi u)_j(p) = \sum_{q \in K} \sum_{i=1}^{c_{\text{in}}} W_{ji}(q)\, u_i(p+q) + b_j$
  ...independent of image size.
- **Stride**: step of the sliding window; stride s downsamples resolution by s.
- **Padding**: extend the domain (zeros/reflection) to control boundary behaviour and output size.
- **Receptive field**: the input region influencing one output value; grows with depth — global structure from local operations.
- **Activation**: Apply σ (e.g. ReLU) pointwise, as in any layer.



One kernel slides over the grid (red arrows); each placement produces one output value (shaded).

Base vs Fibre

$\to \mathbb{Z}^2$ acts on the base only, i.e. weights shared along the base, and stride/padding recoarsen Ω itself *canonically*:
a fixed restriction/sublattice, never learned.
$\to$ The (symmetry-free) fibre instead gets a full dense, *learned* map.

Pooling: from Equivariance to Invariance

Pooling: aggregate over a window $\tilde{K}$, channel by channel

$$\text{maxpool}(u(p)) = \max_{q \in \tilde{K}} (u(p+q)), \qquad \text{avgpool}(u(p)) = \frac{1}{|\tilde{K}|} \sum_{q \in \tilde{K}} u(p+q)$$

$\rightarrow$ Works on the base only, channel-by-channel, also with a stride. Maxpool adds nonlinearity!

Equivariant vs invariant: what the group does to the *output*

$\Phi \circ T_v = T_v \circ \Phi$ (**equivariant**: output moves too) $\qquad$ $\Phi \circ T_v = \Phi$ (**invariant**: it does not).
$\rightarrow$ Invariance is not separate: it is equivariance into the *trivial* representation.

- **Why pool at all**: A pure convolutional stack can only ever be equivariant, so when the answer you want doesn't move with the input — a label, a class, a global invariant — pooling is the only step that actually removes the group direction and delivers it.
- **Local pooling is not invariance**: Stride s merely coarsens $\mathbb{Z}^2$- to $s\mathbb{Z}^2$-equivariance. *Global* average pooling is the invariant one: $u \mapsto \frac{1}{|\Omega|} \sum_p u(p)$.
- **Restriction**: Each stage forgets some fine scale information; conv and pool interleave into a multiscale pyramid, with the translation group restricted to a coarser subgroup each stage.
- **Invariant layers for classification**: Invariance is irreversible, as positional information is lost. For *classification* tasks this is needed at the end to give the label; for *regression* on the base space want equivariant throughout $\longmapsto$ enforcing the hypothesis space symmetry.

Architectures: the development of CNNs

- **LeNet** (1998): conv–pool–conv–pool–MLP on 32×32 digits, $\sim$60k parameters; the blueprint, and it already worked.
- **AlexNet** (2012): the same blueprint at scale (60M parameters, GPUs), plus ReLU, dropout and heavy augmentation; halved the ImageNet error and ignited the field.
- **VGG** (2014): uniform stacks of 3×3 kernels — two stacked 3×3 match one 5×5's receptive field with fewer parameters and an extra nonlinearity.
- **Inception** (2014): parallel 1×1, 3×3, 5×5 branches concatenated — several stencil widths at once, a learned multiscale operator.
- **ResNet** (2015): *residual blocks* $z \mapsto z + F(z)$, plus batch normalisation — layers learn perturbations of the identity, gradients flow through the skip path, $100+$ layers train.
- **U-Net** (2015): encoder–decoder with skip connections — the *equivariant* architecture: fields to fields (segmentation, PDE surrogates), never globally pooled.
- **ConvNeXt** (2022): a ResNet modernised with transformer-era choices (depthwise kernels, LayerNorm, GELU), matching vision transformers.

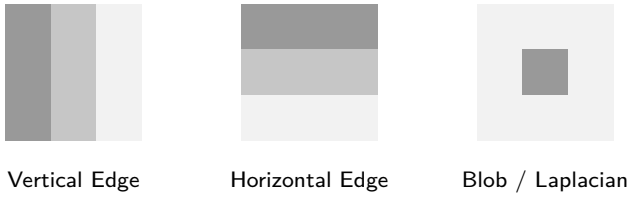
Residual connections, two readings

Optimisation: the identity path prevents vanishing gradients.
Dynamical: $z_{k+1} = z_k + F(z_k)$ is explicit Euler for $\dot{z} = F(z)$ — flows on feature space.

$\rightarrow$ Practical advice: start from a small ResNet (labels) or U-Net (fields); on small scientific data, then develop with more features.

Filters as Local Geometric Operators

- Learned kernels can be *read*: visualise them, correlate with known stencils, see what geometry the network keys on.
- Deeper layers compose these into textures, motifs $\Longrightarrow$ a local-to-global hierarchy of features.



Caricatures of typical learned first-layer filters.

Kernels as stencils

Laplacian kernel: $\Delta_h = A - D = \begin{pmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{pmatrix}$, ...along e_1 only: $\partial_x \simeq \frac{1}{8}\begin{pmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{pmatrix}$

$\rightarrow$ (Sobel): each *row* is the central difference $u(p+e_1) - u(p-e_1)$, the rows weighted $(1, 2, 1)$ to blur in y.
$\rightarrow$ A 3×3 kernel is thus a general 9-point stencil — classical operators are points of $\mathbb{R}^9$.
$\rightarrow$ Δ_h is the grid graph's Laplacian — general graphs give message passing (Lecture 10).

A kernel *is* its moments

$$(w * u)(p) \;=\; m_0\, u(p) \;+\; m_1 \cdot \nabla u(p) \;+\; \tfrac{1}{2}\, m_2 \cdot \nabla^2 u(p) \;+\; \cdots, \qquad m_r := \sum\nolimits_{q \in K} w(q)\, q^{\otimes r}$$

$\rightarrow$ Taylor expansion turns *weights* into *coefficients of a discrete differential operator*.

$\rightarrow$ Impose D_4 (the lattice point group): 9 weights collapse to 3 orbits $w = \begin{pmatrix} b & a & b \\ a & c & a \\ b & a & b \end{pmatrix}$,

...this naturally enforces $m_1 = 0$ and $m_2 \propto I$.

Applications in Geometry — and Non-Applications

Where CNNs fit geometric problems

Any data naturally living on a regular grid with (approximate) translation symmetry.

- **Discretised fields on charts**: curvature, conformal factors, solutions of PDEs sampled on lattices — predict global invariants from local field data.
- **Embedded surface images**: multi-view renderings of surfaces for classification of topological type.
- **Combinatorial data as matrices**: weight matrices of toric varieties / polytope vertex data arranged in arrays — CNNs detected phase-like structure in Calabi–Yau and polytope datasets (with the caveat below).

When *not* to use a CNN

If your data has no grid or no translation structure — point clouds, graphs, sets of polynomial coefficients — the convolution prior is *wrong*.
→ Row order in a matrix of vertices is arbitrary, and a CNN will happily overfit to it!
→ *Match the symmetry of the architecture to the symmetry of the data.*
(Lectures 6 & 10 give the right tools).

PyTorch: a Small CNN

Example (Classifier for $1 \times 64 \times 64$ field images)

```
import torch.nn as nn

model = nn.Sequential(
    nn.Conv2d(1, 16, 3, padding=1), nn.ReLU(),   # (16, 64, 64)
    nn.MaxPool2d(2),                             # (16, 32, 32)
    nn.Conv2d(16, 32, 3, padding=1), nn.ReLU(),  # (32, 32, 32)
    nn.MaxPool2d(2),                             # (32, 16, 16)
    nn.AdaptiveAvgPool2d(1),                     # (32, 1, 1)  global pool
    nn.Flatten(), nn.Linear(32, n_classes)
)
```

- Track shapes in comments at every stage — the discipline that catches most bugs.
- Global average pooling at the end = translation-invariant head (cf. pooling slide).
- Same training loop as Lectures 3–4; on GPU move `model` and batches with `.to(device)`.
- Data augmentation for geometric images: rotations/reflections of the grid (the dihedral group action), doubles as a check of label invariance.

Summary

Key points

- Convolutions are *the* translation-equivariant linear maps. Reducing an MLP to a Convolutional layer is 'weight sharing' which enforces this symmetry.
- Conv layers = learned local operators (general stencils); depth builds receptive fields and local-to-global feature hierarchies.
- Equivariance vs invariance: keep location information while processing, quotient it out later only if the task is invariant (e.g. classification).
- ResNets make depth trainable; residual blocks are discretised flows on feature space.
- Use CNNs when data genuinely lives on a grid with translation structure; otherwise choose an architecture matching the actual symmetry group (Lecture 10).

Next

Tutorial 5: CNNs on discretised curvature fields; visualise learned filters.
Lecture 6: Attention and transformers — architectures for sets and sequences.